The LL(1) Parsing Algorithm

Student Number: 730002704

ECM3428: Algorithms that Changed the World

***Abstract*—This technical report explores the LL(1) parsing algorithm by discussing its principles, complexity, applications, limitations, and implementation. An evaluation of an LLM-generated report is also given.**

I certify that all material in this report which is not my own work has been identified.

Word Count: 1442

I. The LL(1) Algorithm

A. Introduction

a) *Background on Parsing:* A *parser* is a program that verifies if a series of tokens can be generated by a given grammar. Within a compiler, the tokens will come from a lexical analyzer (which in turn generates the tokens from the text of the source code), and the output of the parser provides some kind of structural information about the source program [1].

The LL(1) parsing algorithm is a *top-down*, predictive parser for *context-free grammars*. In the context of parsers, *top-down* refers to how the parsing process begins with some start symbol and attempts to rewrite it until it matches the input string. In this way it generates a *parse tree* that represents the derivation of the input string starting at the root, hence top-down as opposed to some bottom-up parsers that start from the leaves of the parse tree [1].

LL(1) belongs in the family of LL(k) parsers. LL(k) is an acronym for *left-to-right scan*, *leftmost derivation*, with k tokens of lookahead. *Left-to-right scan* means that the parsing occurs by reading input from left to right, and *leftmost derivation* means that the parser always expands the leftmost nonterminal first. This ensures that the parsing is deterministic. The k indicates how many tokens of lookahead the parser uses to make parsing decisions. For LL(1) the parser has only one token of lookahead, which makes it the most efficient in terms of time and space complexity but limits the class of grammars that can be parsed [2].

b) *Grammars:* A grammar is a set of rules, called *productions*, that define how strings in a language can be generated. Each production has left-hand side consisting of a single *nonterminal* symbol, and a right-hand side consisting of a string of *grammar symbols* that can be either *terminal* or nonterminal symbols. Terminal symbols are the basic symbols from which strings are formed (e.g., keywords, operators, identifiers) and nonterminal symbols are placeholders for patterns of terminal symbols that can be further expanded.

A *context-free grammar* is a set of rules that define how strings in a language can be generated that do not depend on the context of a nonterminal. A mathematically rigorous definition of context-free grammars is given by Roskenkrantz et al. [2].

In this report I follow the convention of using lower-case letters (e.g., a, b) to denote terminal symbols, upper-case letters early in the alphabet (e.g., A, B) to denote nonterminal symbols, upper-case letters late in the alphabet (e.g., X, Y) to denote arbitrary grammar symbols (both terminal and nonterminal), and Greek letters (e.g., α, β) to denote strings of grammar symbols. The symbol ϵ denotes the empty string, and S the start symbol. A generic production is therefore $A \to \alpha$. The notation $A \overset{*}{\Rightarrow} \alpha$ means that A can derive α in zero or more steps [1].

B. Principles

a) *Overview:* The LL(1) algorithm is a *predictive* parser. This means that it decides which production to use by looking only at the next input token and the current nonterminal to be expanded. This is in contrast to other parsing methods that may require backtracking or more lookahead tokens.

The algorithm requires an input buffer, a stack, and a parsing table. The input buffer contains the string to be parsed followed by an end-of-input marker (often denoted by \$). The stack contains a sequence of grammar symbols (terminal or nonterminal), initialised with the start symbol of the grammar followed by the end-of-input marker. The parsing table has a row for each nonterminal in the grammar and a column for each terminal, including \$. Each cell of the table contains either a production rule or an error indication. Exactly how this table is constructed is discussed later [1].

b) *Algorithm Overview:* The algorithm starts by intialising a stack with S\$, with S on top. The input buffer is the string to be parsed followed by \$, with the first symbol of the string as the current input token.

Additionally as input the algorithm takes a parsing table constructed from the grammar to be used. The construction of this table can be convoluted but forms a key part of the algorithm and is explained in the next subsection.

Let X be the symbol on top of the stack, and a be the current input token. There are three cases to consider:

1) If $X = a = \$$, then the parsing has been completed successfully.
2) If $X = a \neq \$$, then pop X from the stack and advance the input to the next token.
3) If X is a nonterminal, look up the entry for (X, a) in the parsing table:
    a) If the entry is a production $X \to Y_1 Y_2 ... Y_i$, pop X from the stack and push $Y_i, Y_{i-1}, ..., Y_1$ onto the stack (with Y_1 on top).
    b) If the entry is an error, then the input string is not in the language generated by the grammar.

This process is repeated until either the input is accepted or an error is encountered [1].

c) *Constructing the Predictive Parsing Table:* The construction of a predictive parsing table for an LL(1) grammar requires two functions: *FIRST* and *FOLLOW*.

For any string of grammar symbols α, let $\mathrm{FIRST}(\alpha)$ be the set of terminals that begin the strings derived from α. If $\alpha \overset{*}{\Rightarrow} \epsilon$, then ϵ is also in $\mathrm{FIRST}(\alpha)$. To compute $\mathrm{FIRST}(X)$ for all symbols in a grammar, the following rules are applied iteratively until no more terminals can be added [1]:

1) If X is a terminal, then $\mathrm{FIRST}(X) = \{X\}$.
2) If $X \to \epsilon$ is a production, then add ϵ to $\mathrm{FIRST}(X)$.
3) If X is a nonterminal and $X \to Y_1 Y_2 ... Y_k$ is a production:
    a) Add $\mathrm{FIRST}(Y_i)$ to $\mathrm{FIRST}(X)$ if ϵ is in all $\mathrm{FIRST}(Y_j)$ for $j < i$. That is, if all previous symbols can derive the empty string, then the first terminal of the current symbol can also be the first terminal of the entire production.
    b) If ϵ is in $\mathrm{FIRST}(Y_i)$ for all $i = 1, 2, ..., k$, then add ϵ to $\mathrm{FIRST}(X)$.

First-sets alone are not sufficient to construct the passing table, we also need follow-sets.

Let $\mathrm{FOLLOW}(A)$, for some nonterminal A, be the set of terminals that can appear immediately to the right of A. That

is, the set of terminals a such that there exists a derivation $S \overset{*}{\Rightarrow} \alpha A a \beta$ for some strings α and β. If A can be the rightmost symbol in some sentential form, then the end-of-input marker \$ is also in $\text{FOLLOW}(A)$. $\text{FOLLOW}(A)$ can be computed using the following rules applied iteratively until no more terminals can be added [1]:

1) Place \$ in $\text{FOLLOW}(S)$, where S is the start symbol.
2) If there is a production $A \to \alpha B \beta$, add everything in $\text{FIRST}(\beta)$ except ϵ to $\text{FOLLOW}(B)$.
3) If there is a production $A \to \alpha B$ or $A \to \alpha B \beta$ with ϵ in $\text{FIRST}(\beta)$, then add everything in $\text{FOLLOW}(A)$ to $\text{FOLLOW}(B)$.

From these two functions we can construct a predictive parsing table for some grammar using the following algorithm [1]:

1) For each production $A \to \alpha$ in the grammar:
   a) Add $A \to \alpha$ to the table entry for (A, a) for each terminal a in $\text{FIRST}(\alpha)$.
   b) If ϵ is in $\text{FIRST}(\alpha)$, then add $A \to \alpha$ to the table entry for (A, b) for each terminal b in $\text{FOLLOW}(A)$. Additionally, if \$ is in $\text{FOLLOW}(A)$, add $A \to \alpha$ to the table entry for $(A, \$)$.
2) If any entry in the table is empty, mark it as an error.

A grammar is LL(1) (that is, can be parsed by the LL(1) algorithm) if and only if each cell of the parsing table contains at most one production.

C. Complexity Analysis

For each token of input, the LL(1) algorithm performs a constant amount of work: it either exits, pops a terminal from the stack and advances the input, or looks up a production in the parsing table and adds symbols to the stack. Therefore, if n is the number of tokens in the input string, the time complexity of the LL(1) algorithm is $O(n)$.

D. Applications

As with many algorithms, the LL(1) parsing algorithm makes a trade-off for effciency. By limiting the class of grammars that can be parsed, the algorithm achieves linear time complexity. Consider syntax highlighting in a text editor. The editor needs to parse the source code as the user types, but it is not essential if the parsing is not perfect for incorrect syntax.

Other applications are simple languages like configuration files or domain-specific languages (DSLs) like SQL.

Another advantage of LL(1) parsers is that they can be verified. Lasser et al. [3] present a verified LL(1) parser for JSON that is proven to be correct with respect to the JSON specification.

E. Limitations

The main limitation of the LL(1) parsing algorithm is that it can only handle a subset of context-free grammars. It is therefore not suitable for parsing most complex programming languages like C. Such languages often use a combination of general parsing technqiues and handwritten code to handle ambiguities and context-sensitive constructs. For example, the Rust programming language uses a combination of an LL(3) parser and a custom algorithm to handle ambiguities.

There are algorithms such as Cocke-Younger-Kasami (CYK) and Earley that can parse any context-free grammar, but they are impractical for real use with a worst case time complexity of $O(n^3)$.

A good parser should also be robust and provide meaningful error messages, which can be achieved with an LL(1) parser but at the cost of performance.

II. Pseudo Code

The following pseudo code describes the LL(1) parsing algorithm and the construction of the predictive parsing table.

```
1  procedure Parse(input_buffer, parsing_table)
2      stack ← {$, S}
3      while stack ≠ ∅ do
4          X ← pop(stack)
5          a ← peek(input_buffer)
6          if X = a and a = $ then
7              return true
8          else if X = a then
9              pop(input_buffer)
10         else if X is a nonterminal then
11             rule ← parsing_table[X, a]
12             if rule = error then
13                 return false
14             else
15                 pop(stack)
16                 for symbol in reverse(rule) do
17                     push(stack, symbol)
18                 end for
19         end if
20     end while
21 end procedure
```

```
1  procedure ConstructParsingTable(grammar)
2      parsing_table ← {}
3      for each production A → α in grammar do
4          for each terminal a in FIRST(α) do
5              parsing_table[A, a] ← α
6          end for
7          if ε in FIRST(α) then
8              for each terminal b in FOLLOW(A) do
9                  parsing_table[A, b] ← α
10             end for
11             if $ in FOLLOW(A) then
12                 parsing_table[A, $] ← α
13             end if
14         end if
```

```
    end for
    for each cell in parsing_table do
        if cell is empty then
            cell ← error
        end if
    end for
    return parsing_table
end procedure
```

Appendix

This section was generated entirely by prompting a LLM.

A. Definition

LL(1) is a top-down parsing method for a subset of Context-Free Grammars (CFGs).

- **L**: Scans input from Left to right.
- **L**: Constructs a Leftmost derivation.
- **1**: Uses **1** token of lookahead to determine the next production.

B. Grammar Prerequisites

To be LL(1), a grammar G must satisfy specific structural properties.

a) *1. No Left Recursion:* The grammar cannot contain derivations of the form $A \Rightarrow A\alpha$. Direct left recursion ($A \to A\alpha \mid \beta$) causes an infinite loop in top-down parsers. **Resolution:** Rewrite using right recursion.

b) *2. Left Factored:* The grammar must not have common prefixes in alternatives. If $A \to \alpha\beta_1 \mid \alpha\beta_2$, the parser cannot decide which path to take based on $\text{FIRST}(\alpha)$. **Resolution:** Factor out α into a new non-terminal:

$$\begin{aligned} A &\to \alpha A' \\ A' &\to \beta_1 \mid \beta_2 \end{aligned} \tag{1}$$

C. Lookahead Sets

The parsing table construction relies on two helper functions.

a) *FIRST(α):* The set of terminals that begin strings derived from α.

$$\text{FIRST}(\alpha) = \{a \in \Sigma \mid \alpha \Rightarrow a...\} \cup \{\varepsilon \mid \alpha \Rightarrow \varepsilon\} \tag{2}$$

b) *FOLLOW(A):* The set of terminals that can appear immediately to the right of non-terminal A in some sentential form.

$$\text{FOLLOW}(A) = \{a \in \Sigma \mid S \Rightarrow ...Aa...\} \tag{3}$$

Note: If A is the rightmost symbol, $\$ \in \text{FOLLOW}(A)$ (where \$ is EOF).

D. The LL(1) Condition (Strong LL(1))

A grammar G is LL(1) if and only if for every pair of productions $A \to \alpha \mid \beta$:

1) **Disjoint First Sets:** The first sets of the alternatives must not overlap.
$$\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset \tag{4}$$
2) **Nullable Safety:** If $\beta \Rightarrow \varepsilon$, then α cannot derive any string starting with a terminal in $\text{FOLLOW}(A)$.
$$\varepsilon \in \text{FIRST}(\beta) \Rightarrow \text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset \tag{5}$$

E. Parsing Algorithm

The parser uses a stack, an input buffer, and a parsing table $M[A, a]$.

a) *Parsing Table Construction:* For each production $A \to \alpha$:

1) For each terminal $a \in \text{FIRST}(\alpha)$, add $A \to \alpha$ to $M[A, a]$.
2) If $\varepsilon \in \text{FIRST}(\alpha)$, add $A \to \alpha$ to $M[A, b]$ for each $b \in \text{FOLLOW}(A)$.
3) If $\varepsilon \in \text{FIRST}(\alpha)$ and $\$ \in \text{FOLLOW}(A)$, add $A \to \alpha$ to $M[A, \$]$.

F. Complexity Analysis

The efficiency of LL(1) parsing is split into table construction and runtime parsing.

a) *Time Complexity:*

- **Parsing:** $O(m)$ where m is the length of the input string. Since the grammar is non-left-recursive and left-factored, the parser makes a deterministic decision at each step without backtracking.
- **Table Construction:** $O(N^2)$ generally, or up to $O(N^4)$ depending on the iterative implementation of FIRST and FOLLOW sets, where N is the size of the grammar.

b) *Space Complexity:*

- **Parsing:** $O(D)$ stack space, where D is the maximum depth of the parse tree (proportional to tree height).
- **Table Storage:** $O(|V| \times |\Sigma|)$ for the parsing table M, where V is the set of non-terminals and Σ is the set of terminals. Sparse table representations can reduce this.

G. Applications

While less powerful than bottom-up parsers (like LR), LL(1) is favored for specific use cases due to its simplicity and predictable error reporting.

- **Recursive Descent Prototyping:** LL(1) grammars map one-to-one with hand-written recursive descent parsers, making them ideal for bootstrap compilers.
- **Configuration Languages:** Efficient for parsing linear, hierarchical data formats like JSON or simple config files.
- **IDEs and Tools:** The predictable top-down nature allows for good error recovery and syntax highlighting implementation in lightweight tools.

H. Limitations

LL(1) is a strict subset of Context-Free Grammars, limiting its expressiveness compared to LR(1) or LALR(1).

1) **No Left Recursion:** Requires rewriting the grammar (right recursion), which changes the associativity of operators and can obscure the natural semantic structure.
2) **Limited Lookahead:** Cannot resolve conflicts that require more than 1 token of context.
3) **Ambiguity:** Cannot parse ambiguous grammars. Common constructs like the "dangling else" problem ($S \to$ if E then $S \mid$ if E then S else S) introduce FIRST/FOLLOW conflicts that must be resolved by grammar transformation or ad-hoc precedence rules.

I. Driver Logic (Pseudocode)

```
procedure Parse(input_buffer, parsing_table)
    stack ← [EOF, StartSymbol]
    current_token ← Next(input_buffer)
    while stack.top ≠ EOF do
        X ← stack.top
        a ← current_token.type
        if X is a terminal then
            if X = a then
                stack.pop()
                current_token ← Next(input_buffer)
            else
                Error("Mismatched terminal")
                break
            end if
        else
            M ← parsing_table[X, a]
            if M ≠ Error then
                stack.pop()
                R ← reverse(M)
                for Y in R do
                    if Y ≠ ε then
                        stack.push(Y)
                    end if
                end for
                OutputProductionRule(XtoM)
            else
                Error("No rule found for non-terminal X with lookahead a")
                break
            end if
        end if
    end while
    if current_token.type = EOF then
        Accept
    else
        Error("Stack is empty, but input remains")
    end if
end procedure
```

IV. Details of LLM Generation

A. Prompts

I used Gemini 2.5 Flash to generate the appendix. The prompts used were:

1) You are a computer science final year student. Write the main principles about the LL(1) parsing algorithm. Give your response in the form of Typst code.
2) Add sections for complexity analysis, applications, and limitations.
3) Produce a section of pseudocode to describe the algorithm.

B. Analysis

I feel that the sections describing the algorithm feel rushed and lack depth compared to my report. For example, the grammar prerequesits section does not explain why left recursion and left factoring are necessary. This could make understanding the rest of the report difficult for someone not already familiar with grammars.

The complexity analysis section gives the correct complexity for parsing, but gives a table construction complexity without explaining why it is that complexity. It also provides a space complexity analysis which I did not include in my report.

The applications section gives some good examples that are similar to those that I gave. The limitations section is also good, and identifies the 'dangling else' problem as an example of ambiguity, which I did not mention.

The generated pseudocode is slightly more verbose than mine and I think harder to understand, although it does appear to be correct. No pseudocode was generated for the parsing table construction, which would have been a useful addition.

The generated report does not include any references, which makes it difficult to verify the information given or to read further on the topic.

Overall, I think that the generated report is a good starting point and could be refined with further prompting to add more detail in certain sections. However, it would be difficult to do this (and verify the generated content) without already having good knowledge of the topic.

References

[1] A. V. Aho, R. Sethi, and J. D. Ullman, *Compilers: principles, techniques, and tools*. Addison-Wesley, 2002.

[2] D. Rosenkrantz and R. Stearns, "Properties of deterministic top-down grammars," *Information and Control*, vol. 17, no. 3, pp. 226–256, 1970, doi: 10.1016/S0019-9958(70)90446-8.

[3] S. Lasser, C. Casinghino, K. Fisher, and C. Roux, "A Verified LL(1) Parser Generator," 2012.